MREN 103 – Mechatronics and Robotics Design I
Mechatronics and Robotics Engineering Program
Queen's University, Kingston

Group #30
Laboratory #6 – Bucket Mechanism Design

Student Names: Brady Cabral & Emily Penaranda
Instructor Name: Prof. Surgenor
Date Report Submitted: March 15, 2026

Summary:
Objective of the lab was to design and implement a lift bucket mechanism then to modify the mechanism to lift and curl the bucket. First step shown in **Fig. 1a.** shows the assembled bucket mechanism with lift function using the rotating axle. Next it was placed on the MiniBot with code written to move the bucket automatically to the maximum height. The next step shown in **Fig. 2a.** shows the lift and curl bucket mechanism implemented on the Lego platform, which was then put on the Minibot to automatically cycle the bot from lifting, curling, uncurling and lowering the bucket mechanism with the program shown in **Appendix A.** Finally, shown in **Figure 3 & 4** the linkage program was used to create mocks of the lift and the lift and curl mechanism.

Hardware:
The MiniBot platform was used to place the lift and curl bucket mechanism, which was made up of LEGO pieces. Other hardware components included a servomotor and an Arduino ItsyBitsy microcontroller. The system functioned such that when the servomotor rotated, the linkages in the arm moved the bucked forward, allowing it to lift objects. Similarly, the curl mechanism was implemented by adding an additional linkage between the bucket and the lifring arms, allowing the bucket to rotate as well. The combined lift and curl configuration is shown in **Figure 2a** through **Figure 2c.**

Software:
A listing of the program ***Group30Lab6LiftAndCurl*** used in the lab is given in **Appendix A**. The overall purpose of the program is to test and showcase the functionality of the bucket lift and curl mechanism. The program starts by initializing all the necessary variables, inputs and outputs. Then the program sets the bucket to rest on the ground then waits for the red button to be pressed whilst flashing the green LED, next waits two seconds and then lifts the bucket up to max height, around 10cm. Then the program waits a second and uncurls the bucket then waits a second and curls it back up. Finally, the program pauses for one more second before lowering the bucket back to the ground.

Results*:*

Q1. In the lift mechanism, a singularity takes place when the linkage arms become aligned in a way in which they can no longer generate motion effectively, losing mechanical advantage. Three pictures were taken to illustrate the singularity, namely at rest position, at the singularity and past the singularity, as shown below:

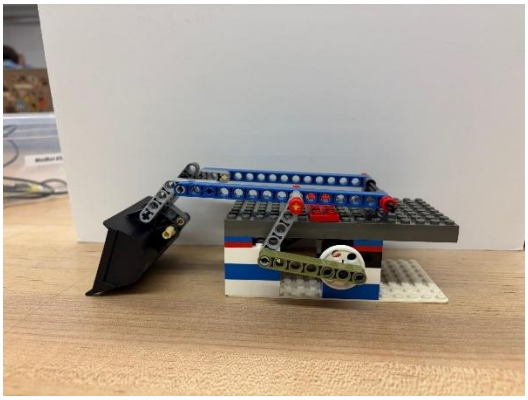

Fig 1a. Initial rest position of bucket mechanism on Lego platform

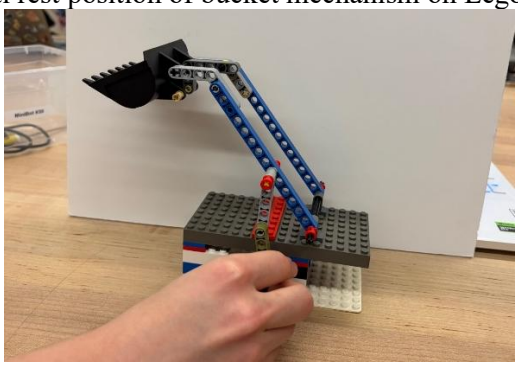

Fig 1b. Singularity position of simple bucket mechanism on Lego platform

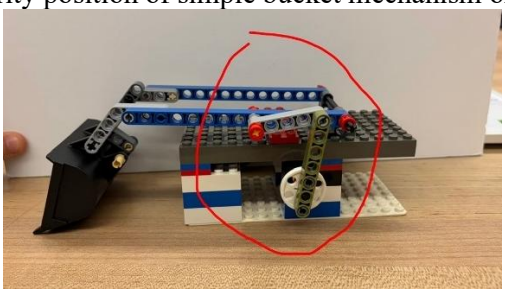

Fig 1c. Post singularity rest position of simple bucket mechanism on Lego platform

The negative effects of singularity can be mitigated effectively in several ways. For example, limiting the servomotor's range so that it never reaches the singularity will prevent this from happening at all when operating the lift motion. Adjusting the position or length of the linkage arms, as well as changing the mounting point of the servomotor could also mitigate the occurrence of singularities while operating.

Q2. In the curling mechanism, a potential singularity could occur when the bucket linkage becomes aligned with the arms linkage in such a way that the bucket cannot rotate effectively anymore. As with the lift mechanism, three pictures were taken, one at resting position, one at the singularity point, and another past the singularity, as shown below:

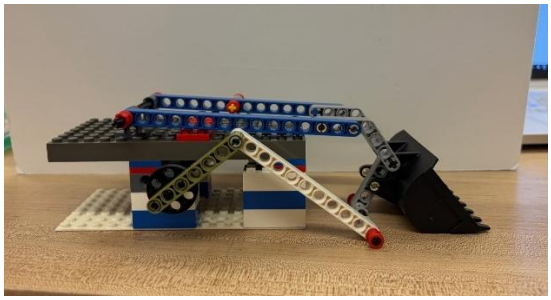

Fig 2a. Initial rest position of lift and curl bucket mechanism on Lego platform

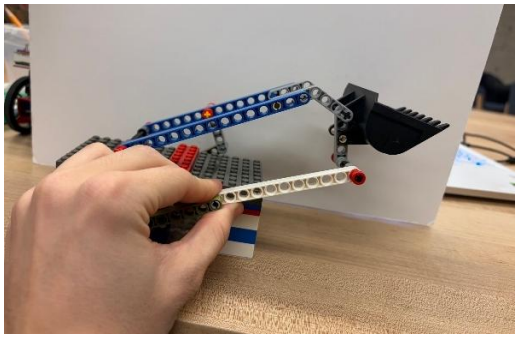

Fig 2b. Singularity position of lift and curl bucket mechanism on Lego platform

Fig 2c. Post singularity rest position of lift and curl mechanism on Lego platform

To mitigate the negative effects of the singularity, the servomotor range can also be limited. Another way could also be to introduce an additional link to redistribute motion across the mechanism. It is also worth noting that the lift mechanism can interact with the curling mechanism. For example, lifting the bucket can change the geometry of the curl linkage, bringing the latter closer to a singularity. For this reason, it is important to make sure that both systems work as intended and without singularities, also conducting tests to ensure smooth operation in tandem.

Conclusions:

In this lab, a bucket lift and curling mechanism was implemented on the MiniBot with LEGO linkage arms and a servomotor. A key concept introduced in this lab was singularities, which represent a linkage alignment that prevents effective force transmission. By analyzing and adjusting the linkage geometry and the servomotor's motion range, the robot was able to perform the intended lift and curl motions without encountering singularity configurations.

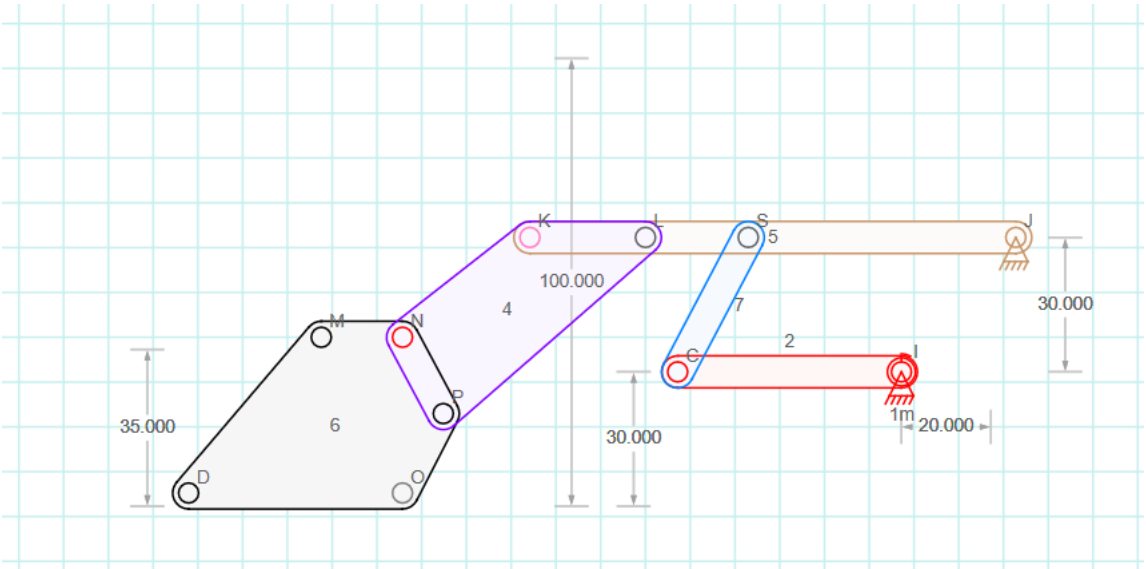

Fig 3. Simple lift mechanism shown in linkage software

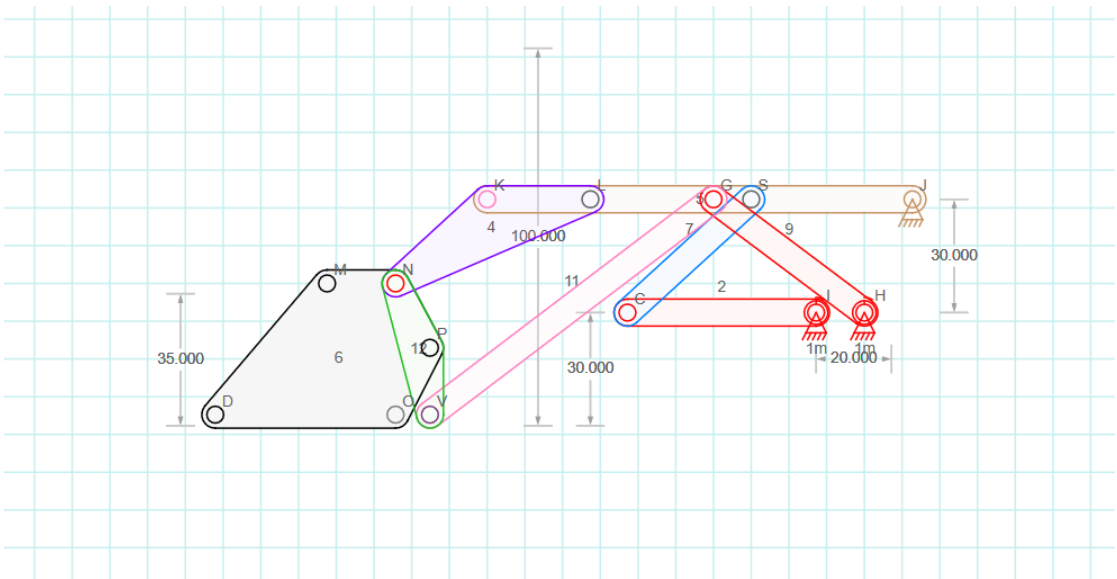

Fig 4. Lift and curl mechanism shown in linkage software

Appendix A - Program Listing:

```
/*****************************************************
Group30Lab6LiftAndCurl (v1.2)

  Original by D. Ells 27/8/2021
  Revised by B. Surgenor, 25/04/2021
   Updated by B. Cabral & Emily Vargas-Penaranda,
15/02/2026

Lift, uncurl, curl and then lower the bucket once. Start
with
bucket resting (at an angle) touching ground, drive
link parallel to the ground.
*****************************************************/

#include <Servo.h>  // Includes the library
Servo myServoA;  // Makes a servo object to control
servo A
Servo myServoB;

// Pin Assignments
int GRN = 9;          // Green LED Pin
int YLW = 5;          // Yellow LED Pin
int RED = 10;         // Red LED Pin
int BUTTON = 7;       // Pushbutton Pin

int servoPinA = 11;     // Bucket servomotor #1 pin
int servoPinB = 12;
int myAngleA1 = 140;    // initial angle, bucket lifts off
ground if too high
int posA = myAngleA1;   // if set to 180, bucket lifts robot
off of ground
int myAngleA2 = 85;     // highest angle (lift), puts almost
straight, set to 110
                    //    still bent (i.e. not as high)
int myAngleB1 = 90; // good start
int posB = myAngleB1;
int myAngleB2 = 140;

// Set-up routine
void setup() {

// Set-up LED pins as outputs
  pinMode(GRN, OUTPUT);
  pinMode(YLW, OUTPUT);
  pinMode(RED, OUTPUT);

// Set-up button pin as input
  pinMode(BUTTON, INPUT);

  myServoB.write(posB);
  myServoB.attach(servoPinB);

// Set-up servo motors
  myServoA.write(posA);       // Servo A starting position
  myServoA.attach(servoPinA);   // Attaches the servo to
the servo object
}

// Main routine
void loop() {
turnOnLED(GRN);
do {
   toggleLED(GRN);                  // Toggle green LED on
} while(digitalRead(BUTTON)== LOW);    // Press button
to start

   turnOnLED(YLW);                  // Yellow LED on

delay (2000);          // A couple seconds to stand back
  for (posA = myAngleA1; posA >= myAngleA2; posA--)
{ // Lift action
   myServoA.write(posA);
   delay(20);
  }

delay(1000);
for (posB = myAngleB1; posB <= myAngleB2; posB++) {
  myServoB.write(posB);
  delay(20);
}

delay(1000);
for (posB = myAngleB2; posB >= myAngleB1; posB--) {
  myServoB.write(posB);
  delay(20);
}

delay(1000);
  for (posA = myAngleA2; posA <= myAngleA1; posA++)
{  // Drop action
   myServoA.write(posA);
   delay(20);
  }
}

//********************                                Functions
(subroutines)****************

//Toggle an LED on/off
void toggleLED(int colour){
  digitalWrite(colour, HIGH);
  delay(250);
  digitalWrite(colour, LOW);
  delay(250);
}

// Turn on a single LED and turn others off
void turnOnLED(int COLOUR)
{
  digitalWrite(GRN, LOW);
  digitalWrite(YLW, LOW);
  digitalWrite(RED, LOW);
  digitalWrite(COLOUR, HIGH);
}
```